Languages and Strings:

Alphabet: * An alphabet is a finite set of symbols/characters
* Represented using Σ.

Ex: $\Sigma = \{0, 1\}$, binary alphabet.

$\Sigma = \{a, b, \ldots z\}$, English alphabet etc.

Strings: * A string is a finite sequence, possibly empty, of symbols drawn from some alphabet Σ.
* The shortest string is an empty string, denoted by 'ϵ' or 'ε' (epsilon).

Ex: 0, 1, 001, 1010, 011001 etc.

abcg, daabcd, zhyc etc.

* The set of all possible strings over an alphabet Σ is written Σ^*, called as star closure/ Kleene closure.

Functions on Strings:

(1) Length of a string 's', $|s|$ is the number of symbols in s.

eg: $|010110| = 6$

$|\epsilon| = 0$

Number of times, the symbol 'c' occurs in 's' can be defined using a function, $\#_c(s)$

Ex: $\#_0(010110) = 3$.

(2) Concatenation of two strings: s and t, written s||t or s.t is the string formed by appending t to s.

eg: x = hello, y = world, then x.y = helloworld

ϵ is the identity for concatenation of strings.

$\therefore \boxed{\forall x \ (x\epsilon = \epsilon x = x)}$

eg: $101 \cdot \epsilon = \epsilon \cdot 101 = 101$ where 101 is a string.

Concatenation on strings is associative

eg: $x = 101$ $y = 1110$ $z = 001$

then $x \cdot (yz) = (xy)z = 1011110001$

ie, $\boxed{\forall s, t, w, ((st)w = s(tw))}$

(3) Replication:

For each string 'w', and each natural number i, the string w^i is defined as:

$$\boxed{\begin{aligned} w^0 &= \epsilon \\ w^{i+1} &= w^i w \end{aligned}}$$

Ex: $a^4 = aaaa$

$(hello)^2 = hellohello$

$a^0 b^2 c^3 = \epsilon \cdot bbccc = bbccc$

(4) Reversal:

For any string w, the reversal is written as w^R. It is defined as,

If $|w| = 0$, then $w^R = w = \epsilon$

If $|w| \geq 1$, then $\exists a \in \Sigma_1 (\exists u \in \Sigma_1^* (w = ua))$,

Then define $w^R = au^R$

eg: $w = abaac$, then $w^R = caaba$

$w = \epsilon$, then $w^R = \epsilon$

Theorem 2.1 Concatenation and Reverse of Strings

If w and x are strings, then $(wx)^R = x^R w^R$

Proof: The proof is by induction on $|x|$

Base: $|x| = 0$, Then $x = \varepsilon$.

$(wx)^R = (w\epsilon)^R = (w)^R = (\varepsilon w)^R = \varepsilon^R w^R = x^R w^R$

Prove: $\forall n \geq 0$

$(((|x| = n) \rightarrow ((wx)^R = x^R w^R))$
$\rightarrow ((|x| = n+1) \rightarrow ((wx)^R = x^R w^R)))$.

Let x be any string, where $|x| = n+1$

Then $x = ua$ for some character a and $|u| = n$

So,

$$
\begin{aligned}
(wx)^R &= (w(ua))^R && \text{// Associativity of concatenation} \\
&= ((wu)a)^R && \text{// definition of reversal} \\
&= a(wu)^R && \text{// Induction hypothesis} \\
&= a(u^R w^R) && \text{// Associativity of concatenation} \\
&= (au^R)w^R && \text{// definition of Reversal} \\
&= (ua)^R w^R && \text{// rewrite } ua \text{ as } x. \\
&= x^R w^R
\end{aligned}
$$

Relations on strings

A string 's' is a substring of a string 't' iff s occurs contiguously as part of t.

eg: aaa is a substring of baabaaabb

aaa is not a substring of aabbbabab

- A string s is a proper substring of a string t iff s is a substring of t and $s \neq t$.

The empty string, ε

- A string 's' is a <u>prefix</u> of t iff $\exists x \in \Sigma^* (t = sx)$.
  Every string is a prefix of itself
  Eg: Suppose $t =$ aabaa,
  then the prefixes are ε a, aa, aab, aaba, aabaa

<u>NOTE:</u> The empty string is a <u>prefix</u> of every string.

- A string 's' is a <u>proper prefix</u> of a string t iff s is a prefix of t and $s \neq t$.

  Eg: Suppose $t =$ aabaa,
  then the proper prefixes are: ε, a, aa, aab, aaba

- A string 's' is a <u>suffix</u> of t iff $\exists x \in \Sigma^* (t = xs)$.
  Every string is a suffix of itself.
  The empty string ε, is a <u>suffix</u> of ~~itself~~ every string.

  Eg: Suppose $t =$ aabaa
  then the suffix of t are: ε, a, aa, baa, abaa, aabaa

- A string 's' is a <u>proper suffix</u> of a string t iff s is a suffix of t and $t \neq s$.

  <u>Eg:</u> Suppose $t =$ aabaa,
  then the propersuffix of t are: ε, a, aa, baa, abaa.

<u>Languages:</u>

- A Language is a (finite or infinite) set of strings over a finite alphabet Σ.

Notation: Σ_L : Alphabet from which the strings in the language L are formed.

Example: Let $\Sigma = \{a, b\}$

$\Sigma^* = \{\varepsilon, a, b, aa, ab, ba\ aab \ldots\}$

$L_1 = \{\varepsilon\}$ ⇒ Language containing empty string

$L_2 = \phi$ ⇒ Empty / Null language.

$L_3 = \{ab, ba, aa, bb\}$ ⇒ Language of strings whose length is 2

Techniques for defining languages:

a) Specify a characteristic function, ie, a predicate that is True of every element in the set and False of everything else.

Ex:1: $L = \{ w \in \{a,b\}^* :$ all a's precede b's in $w\}$

ie, $L = \{\varepsilon, a, aa\ aabbb, bb\}$

Ex:2: $L = \{ x : \exists y \in \{a,b\}^* (x = ya)\}$, The strings that end in 'a'.

eg. $L = \{a, aa, ba, aba \ldots$ etc$\}$

whereas ε, ab, bcd etc do not belong to this language

NOTE: In the above, $L_1 \neq L_2$

Ex:3: $L = \{ w : w$ is a sentence in English$\}$] Complex, special specification is needed

Ex 4: A halting Problem Language.

$L = \{ w : w$ is a C program that halts on all inputs$\}$.

b) Using the prefix relation:

Language can be specified in terms of the prefix relation on strings

Ex1: $L_1 = \{ w \in \{a,b\}^* :$ no prefix of w contains b

ie, $L_1 = \{ \epsilon, a, aa, aaa, aaaa, \ldots \}$

Ex2: $L_2 = \{ w \in \{a,b\}^* :$ no prefix of w starts with $b \}$

ie, $L_2 = \{ w \in \{a,b\}^* :$ the first character of w is $a \} \cup \{\epsilon\}$

Ex3: $L_3 = \{ w \in \{a,b\}^* :$ Every prefix of w starts with $b \}$

$= \phi$

Here, ε is a prefix of every string and since ε does not start with b, no strings meet L_3's requirement.

c) <u>Using Replication to define a Language</u>

Ex1: $L_1 = \{ a^n : n \geq 0 \}$

ie, $L_1 = \{ \varepsilon, a, aa, aaa, \ldots \}$

Ex2: $L_2 = \{ a^n b^n : n \geq 1 \}$

ie, $L_2 = \{ ab, aabb, aaabbb \ldots \}$

d) <u>Computational definition:</u>

A language can be specified as

* language generator – Enumerates (lists) the elements of the language.

* language recognizer – Returns True if candidate string is in the language, and returns False if it isn't.

- When considering an enumerator for a language L, we may specify <u>lexicographic order</u> for the elements in the set such that:
  * Shorter strings precede longer ones
    $$\forall x(\forall y((|x| < |y|) \rightarrow (x <_L y)))$$
  * Strings of same length can be sorted in a dictionary order.

- We say that a program <u>lexicographically enumerates</u> the elements of L iff it enumerates them in lexicographic order.

- <u>Ex:</u> $L = \{ x \in \{a,b\}^* : \text{all a's precede all b's} \}$
  The lexicographic enumeration of L is:
  $\{\varepsilon, a, b, aa, bb, aaa, abb, bbb, aaaa, aaab \ldots\}$

<u>Cardinality of a language:</u>

- How large a language is, defined in terms of its cardinality.
- The smallest language over any alphabet Σ is ϕ, whose cardinality is 0.
- The largest language over any alphabet Σ is Σ^*.
  If $\Sigma = \phi$, then $\Sigma^* = \{\varepsilon\}$ and $|\Sigma^*| = 1$.
  <u>If $\Sigma \neq \phi$, Then Σ^* is countably infinite.</u> [Theorem]

  <u>Proof:</u> The elements of Σ^* can be lexicographically enumerated
  * Enumerate all strings of length 0, length 1 and, length 2, and so on.

* Within the strings of a given length, enumerates them in dictionary order.

This enumeration is infinite since there is no longest string in Σ^*.

By theorem (A.1)

Since there exists an infinite enumeration of Σ^*, it is countably infinite.

Since any language over Σ is a subset of Σ^*, the cardinality of every language is at least 0 and atmost $\aleph_0$.

So all languages are either finite or countably infinite.

How many languages are there?

The set of languages defined on Σ is $P(\Sigma^*)$, the power set of Σ^*, or the set of all subsets of Σ^*.

If $\Sigma = \phi$, $\Sigma^* = \{\epsilon\}$ and $P(\Sigma^*) = \{\phi, \{\epsilon\}\}$

If $\Sigma \neq \phi$, then the set of languages over Σ is uncountably infinite.

Proof: The set of languages defined on Σ is $P(\Sigma^*)$

Also, from the previous theorem, Σ^* is countably infinite.

From theorem A.4, if S is countably infinite set, $P(S)$ is uncountably infinite.

$\therefore P(\Sigma^*)$ is uncountably infinite.

Suppose, $\Sigma = \{a, b\}$

$\Sigma^0 = \phi$

$\Sigma^1 = \{a, b\} = L_1$

$\Sigma^2 = \{a, b\}.\{a, b\} = \{aa, ab, ba, bb\} = L_2$

$\Sigma^3 = \Sigma^2 . \Sigma = \{aa, ab, ba, bb\}.\{a, b\}$

$= \{aaa, aba, baa, bba, aab, abb, bab, bbb\}$

$\vdots$

$\Sigma^* = \{\Sigma^0\} \cup \{\Sigma^1\} \cup \ldots\ldots\ldots\}$ Infinite

$= \{\ldots\ldots\}$ infinite, since there is no longest string in Σ^*

NOTE:

(1) A set is countably infinite if one can count off all elements in the set in such a way that, even though the counting will take forever, you will get to any particular element in a finite amount of time.

(2) To stress that we are excluding finite sets, we usually use the term countably infinite.

Ex: $\{0, 1, -1, 2, -2, 3, -3, \ldots\}$ is an infinite set. [Integers]

We can count off all integers.
However, if we say $-20, 342, 876, 748$, we will get to this integer in the counting process in a finite amount of time.

(3) Uncountable infinite implies, the set is so large that it cannot be counted even if we kept counting forever.
Ex: Set of Real Numbers.

Functions on Languages:

(1) Concatenation:

Let L_1 and L_2 be two languages defined over some alphabet Σ. Then their concatenation, written $L_1 \cdot L_2$ is:

$$L_1 \cdot L_2 = \{ w \in \Sigma^* : \exists s \in L_1 (\exists t \in L_2 (w = st)) \}.$$

Ex: $L_1 = \{a, b, c\}$

$L_2 = \{aa, bb\}$

Then, $L_1 \cdot L_2 = \{aaa, abb, baa, bbb, caa, cbb\}$

* The language $\{\varepsilon\}$ is an identity for concatenation of languages.

ie, For all L, $L \cdot \{\varepsilon\} = \{\varepsilon\} \cdot L = \{L\}$

eg: $L = \{aa, bb\}$

$$L \cdot \{\varepsilon\} = \{aa, bb\} \cdot \{\varepsilon\}$$
$$= \{aa, bb\}$$
$$= L$$

* The language ϕ is a zero for concatenation of languages.

ie, for all L, $L \cdot \phi = \phi \cdot L = \phi$.

eg: $L = \{aa, bb\}$

$$L \cdot \phi = \{aa, bb\} \cdot \phi = \phi$$

* Concatenation, as a function defined on languages is associative.

ie, $(L_1 L_2) L_3 = L_1 (L_2 L_3)$

* Careful w.r.t. concatenating replication languages.

Ex: Suppose $L_1 = \{a^n \mid n \geq 0\}$ and $L_2 = \{b^n \mid n \geq 0\}$

then, $L_1 \cdot L_2 = \{a^n b^m \mid n, m \geq 0\}$ [NOTE: here, the superscript name for L_2 is changed to m]

(2) Kleen star (Star closure) of L:

The Kleen star of L, written as L^* is defined as,

$$L^* = \{\varepsilon\} \cup \{w \in \Sigma^* : \exists k \geq 1 (\exists w_1, w_2, \ldots w_k \in L (w = w_1 w_2 \ldots w_k))\}.$$

ie, L^* is the set of strings that can be formed by concatenating together zero or more strings from L.

Ex: $L = \{a, b\}$

$L^* = \{\varepsilon\} \cup L^1 \cup L^2 \cup \ldots\ldots$

where $L^1 = \{a, b\}$ # strings made up of length = 1

$L^2 = \{aa, ab, ba, bb\}$ # strings of length = 2

⋮

and so on.

$\therefore L^* = \{\varepsilon, a, b, aa, ab, ba, bb \ldots\ldots\}$

Special cases:

If $L = \phi$, then $L^* = \{\varepsilon\}$

If $L = \{\varepsilon\}$, then $L^* = \{\varepsilon\}$

Positive closure:

Is written as L^+ is defined as,

$$L^+ = L L^*$$

ie, set of strings that can be formed by concatenating together one or more strings from L.

Ex: $L = \{a, b\}$

$L^+ = L^1 \cup L^2 \ldots\ldots$

ie, $L^+ = \{a, b, aa, ab, \ldots\ldots\}$

NOTE: Here, L^+ doesn't include ε

(3) Reversal of a language

Let L be a language defined over some alphabet Σ. Then the reverse of L, written as L^R is:

$$L^R = \{ w \in \Sigma^* \mid w = x^R \text{ for some } x \in L \}$$

ie, L^R is the set of strings that can be formed by taking some string in L and reversing it.

Ex: $L = \{ aab, \ abb, \ b, \ bbb \} = \{ w \in \{a,b\}^*, \ x = wb \}$

$L^R = \{ baa, \ bba, \ b, \ bbb \}$

(4) Difference:

Let L_1 and L_2 be two languages defined over some alphabet Σ. Then, their difference, written as $L_1 - L_2$ is: $\{ w \in \Sigma^* : \exists s \in L_1 (\exists t \in L_2 (w = s ; s \neq t)) \}$

Ex: $L_1 = \{ ab, \ bb, \ bbb \}$

$L_2 = \{ ab, \ aa, \ aaa \}$

$L_1 - L_2 = \{ bb, \ bbb \}$

(5) Intersection:

Let L_1 and L_2 be two languages defined over some alphabet Σ. Then, their intersection written as,

$L_1 \cap L_2 = \{ w \in \Sigma^* : \exists s \in L_1 (\exists t \in L_2 (w = s : s = t)) \}$

Ex: $L_1 = \{ ab, \ bb, \ bbb \}$

$L_2 = \{ ab, \ aa, \ aaa \}$

$L_1 \cap L_2 = \{ ab \}$

(6) Complement:

Let L be a language, Then, the complement operation is defined as $\underline{L^c}$ or $\underline{\neg L}$, the set of all strings that doesn't belong to L.

Ex: $L = \{$ Strings with even number of a's $\}$ over $\Sigma = \{a\}$

ie, $L = \{\varepsilon, aa, aaaa, aaaaaa \ldots\}$

If L^* contains all the strings made up of a's,

then, $\underline{L^c}$ or $\underline{\neg L} = L^* - L$

$= \{a, aaa, aaaaa, \ldots\}$

$= \{$ Set of odd number of a's in a string $\}$

Theorem: If L_1 and L_2 are languages, then $(L_1 L_2)^R = L_2^R \cdot L_1^R$

Proof: If x and y are strings, then

$\forall x(\forall y((xy)^R = y^R x^R))$ - Reversal of strings

$(L_1 L_2)^R = \{(xy)^R : x \in L_1 \text{ and } y \in L_2\}$ - definition of Concatenation of languages.

$= \{y^R x^R : x \in L_1 \text{ and } y \in L_2\}$ - Lines 1 and 2

$= L_2^R \cdot L_1^R$ - Definition of Concatenation of languages.